UADE

Exposición de experto

Temas a desarrollar

- Operaciones de las clases
- Cardinalidad
- Direccionalidad
- Ordinalidad
- Síntesis

Operaciones de las clases

Relaciones entre objetos

Los objetos se comunican, se relacionan entre sí, **enviándose mensajes.**

Cuando los objetos intercambian mensajes para realizar una operación, el objeto que recibe el mensaje responde enviando a su vez otro mensaje que contiene el resultado de la operación ejecutada.

En este proceso de intercambio de mensajes se establecen relaciones, basándose en tres formas fundamentales:

- **1. Cardinalidad.**
- **2. Direccionalidad.**
- **3. Ordinalidad.**

Cardinalidad

Concepto

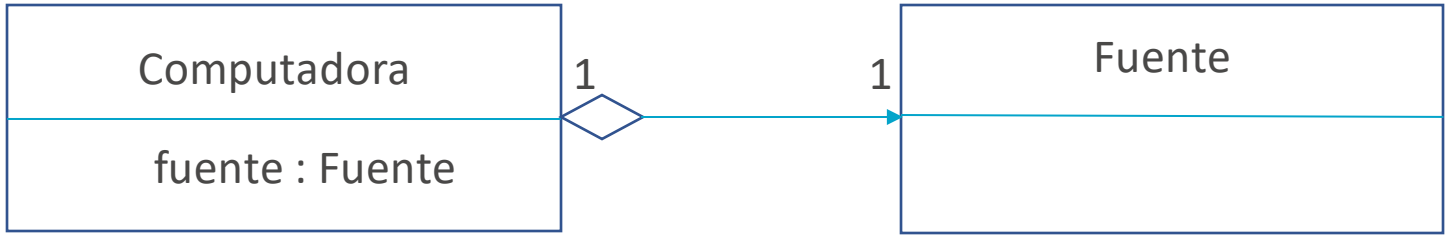
Da cuenta de cuántos objetos existen de cada lado de la relación.

Considerando una relación binaria entre el juego de entidades **A** y el **B**, la cardinalidad puede ser:

- ✓ **UNO a UNO.**
- ✓ **UNO a MUCHOS, o MUCHOS a UNO.**
- ✓ **MUCHOS a MUCHOS.**

UNO a UNO

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:



Ejemplo de código: una Computadora tiene una sola Fuente.

Computadora.java

```
public class Computadora {  no usages
    //Relacion 1 a 1 con Fuente
    Fuente fuente;  no usages
    String marca;  no usages
    String modelo;  no usages
}
```

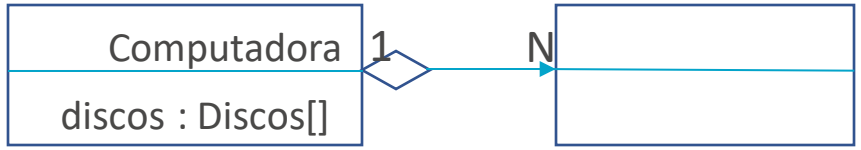
Fuente.java

```
public class Fuente {  1 usage
    private String Marca;  no usages
    private String Modelo;  no usages
    private int Potencia;  no usages

}
```

UNO a MUCHOS

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:



Ejemplo de código: una computadora puede tener un o más discos rígidos.

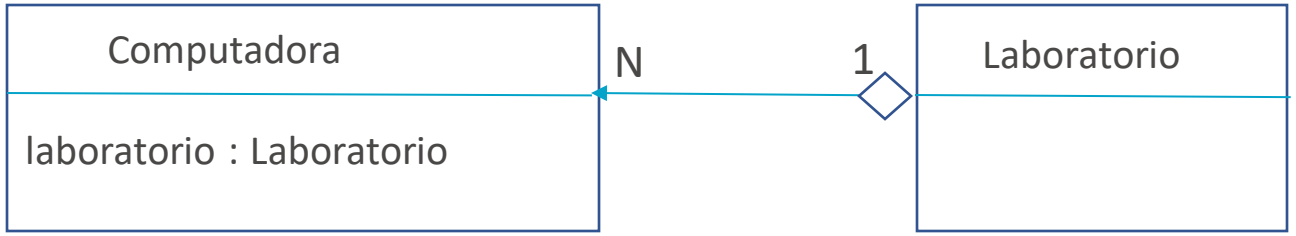
Computadora.java

```
public class Computadora {  no usages
    //Relacion 1 a 1 con Fuente
    Fuente fuente;  no usages
    String marca;  no usages
    String modelo;  no usages

    //Podemos tener un array con la cantidad de discos
    //que tenga la computadora
    Discos[] disco;  no usages
}
```

MUCHOS a UNO

Una entidad en A está asociada con cualquier número (0 o más) de entidades en B:



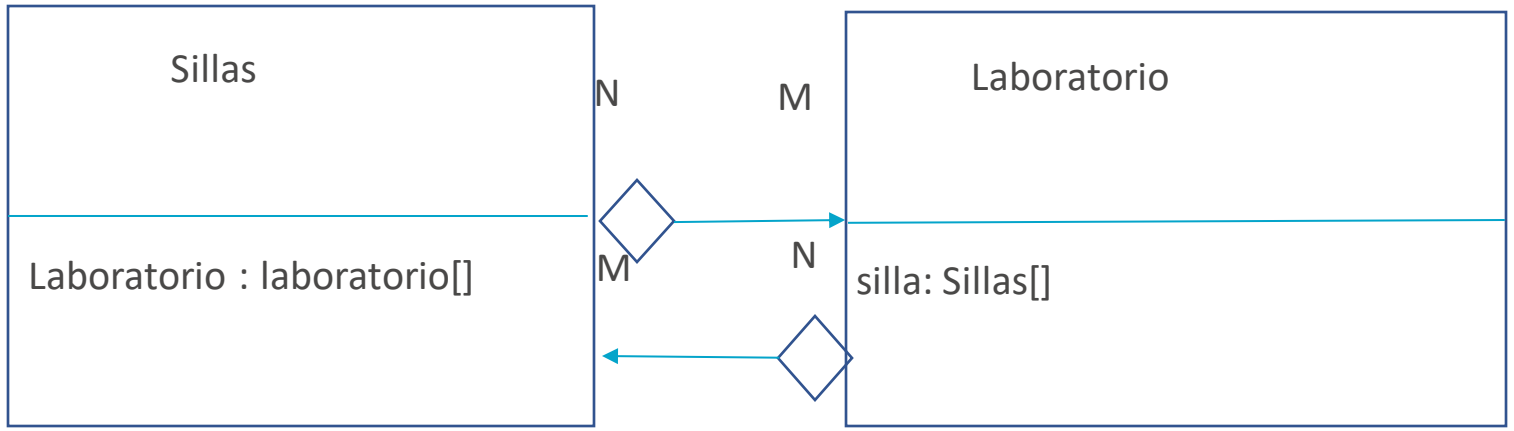
Computadora.java

```
public class Computadora {  no usages
    //Una computadora va a estar instalada solo
    //1 laboratorio. Pero ese laboratorio puede
    //tener instaladas muchas compputadoras
    Laboratorio laboratorio;     no usages
```

Ejemplo de código: se supone que una computadora está instalada en un laboratorio, pero que ese laboratorio puede tener N cantidad de computadoras.

MUCHOS a MUCHOS

Una entidad en A está asociada con cualquier número de entidades en B (0 o más) y viceversa:



MUCHOS a MUCHOS

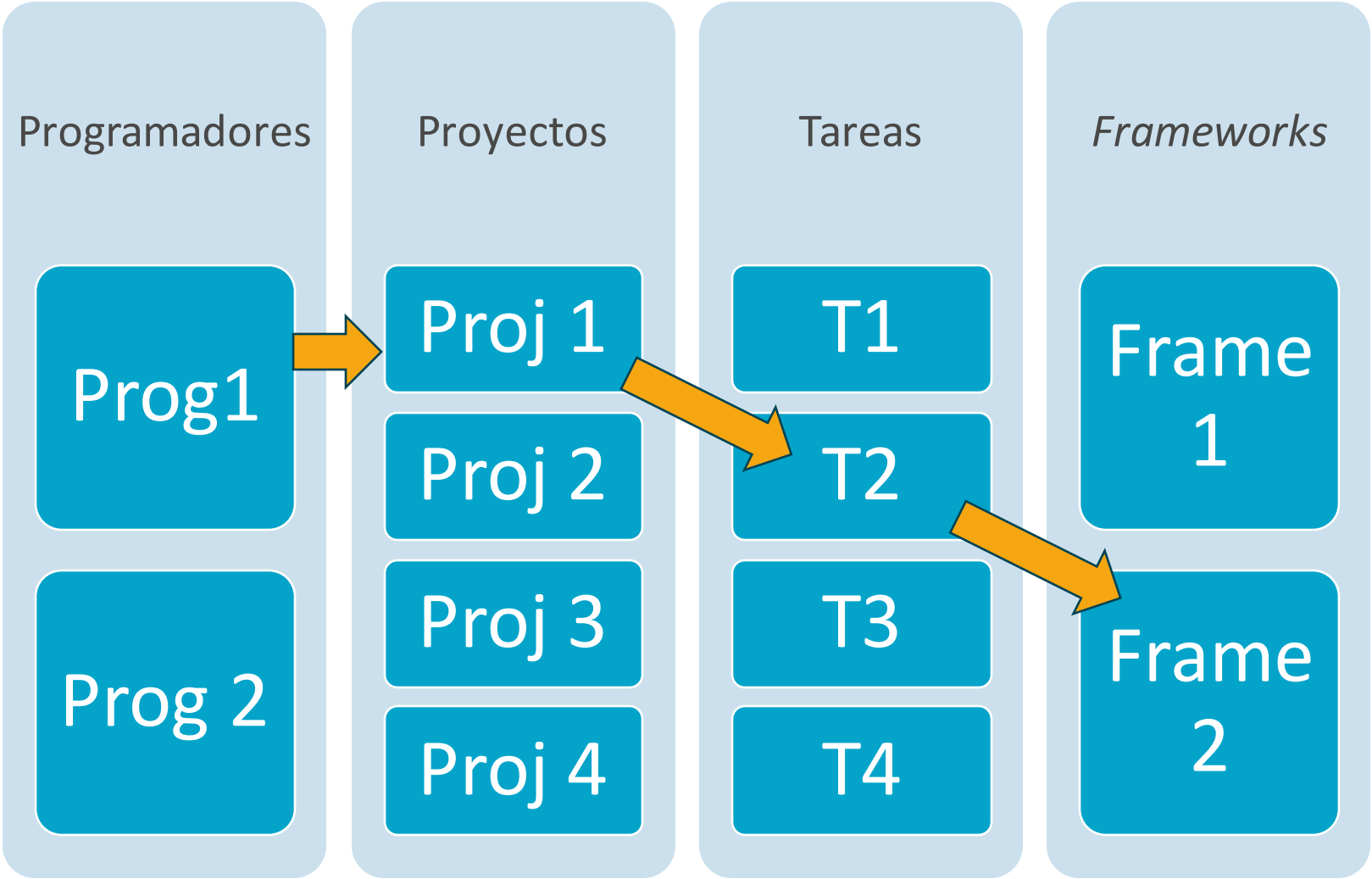
Ejemplo de código I: las sillas van a tener asignado un laboratorio. Estas se podrían rotar dentro de los laboratorios en forma cíclica.

Sillas.java

```java
public class Sillas {  no usages
    private String numeroSilla;  no usages
    private Laboratorio laboratorios[];  no usages
}

```

Concepto


Programadores
Proyectos
Tareas
Frameworks
Prog1
Prog 2
Proj 1
Proj 2
Proj 3
Proj 4
T1
T2
T3
T4
Frame 1
Frame 2

Direccionalidad

Concepto

RELACIÓN UNIDIRECCIONAL: para cada objeto de un lado de la relación le corresponden uno o más objetos del otro lado de la relación.

RELACIÓN BIDIRECCIONAL: para cada objeto de un lado de la relación le corresponden muchos objetos del otro lado, **y viceversa.**

Relación unidireccional

Ejemplo-1: En un sistema de gestión de una biblioteca, se pueden tener diferentes tipos de usuarios, como estudiantes, profesores y personal administrativo. Cada usuario puede tener un conjunto de libros prestados en un momento dado. El sistema debe ser capaz de registrar qué libros tiene prestados cada usuario y permitir la devolución de los mismos.

Ejemplo-2: En una empresa de desarrollo de *software*, cada programador puede tener asignado un conjunto de proyectos en los que trabaja. Sin embargo, la empresa solo necesita saber qué proyecto tiene asignado cada programador en un momento determinado, en lugar de mantener un registro detallado de todos los proyectos disponibles.

Relación bidireccional

Ejemplo: En una empresa de desarrollo de software, se dispone de un listado completo de programadores. La empresa necesita poder consultar dos tipos de información:

1) Saber qué proyectos maneja un programador específico.

2) Saber qué programador está asignado a un proyecto particular.

Se debe implementar un sistema de seguimiento que asocie a cada programador con los proyectos en los que trabaja en un momento dado, así como también registre qué programador está asignado a cada proyecto. .

Ordinalidad

Concepto - Agregación

La agregación y la composición son dos tipos de relaciones entre objetos en un sistema orientado a objetos que indican la importancia y la dependencia entre ellos.

- **Agregación:** un objeto (llamado el objeto contenedor) "usa" otro objeto (llamado el objeto contenido) para realizar alguna funcionalidad, pero el objeto contenido no es propiedad exclusiva del objeto contenedor y puede existir independientemente de él. Esta relación es más débil y flexible. Un ejemplo podría ser una relación entre una biblioteca y un libro. La biblioteca puede contener varios libros, pero los libros pueden existir fuera de la biblioteca.

Concepto - Composición

- **Composición:** un objeto (llamado el objeto compuesto) es "dueño de" otro objeto (llamado el objeto componente), y el objeto componente no tiene razón de existir sin el objeto compuesto. Esta relación es más fuerte y más rígida. Un ejemplo sería la relación entre una casa y sus habitaciones. Las habitaciones existen solo dentro de la casa y no tienen sentido fuera de ella.

En resumen, la diferencia principal entre la agregación y la composición radica en el grado de dependencia entre los objetos. La composición implica una relación más fuerte y una dependencia más estrecha, mientras que la agregación implica una relación más débil y una dependencia más flexible.

Ejemplo AGREGACIÓN

Ejemplos:

- ✓ Una **computadora tiene una Fuente.**
- ✓ Un **Auto tiene un Motor** que le permite funcionar

Ejemplo de código agregación:

Computadora.java

```
1
2  public class Computadora {  no usages
3      //Relacion 1 a 1 con Fuente
4      Fuente fuente;  no usages
5
```

Ejemplo de COMPOSICIÓN

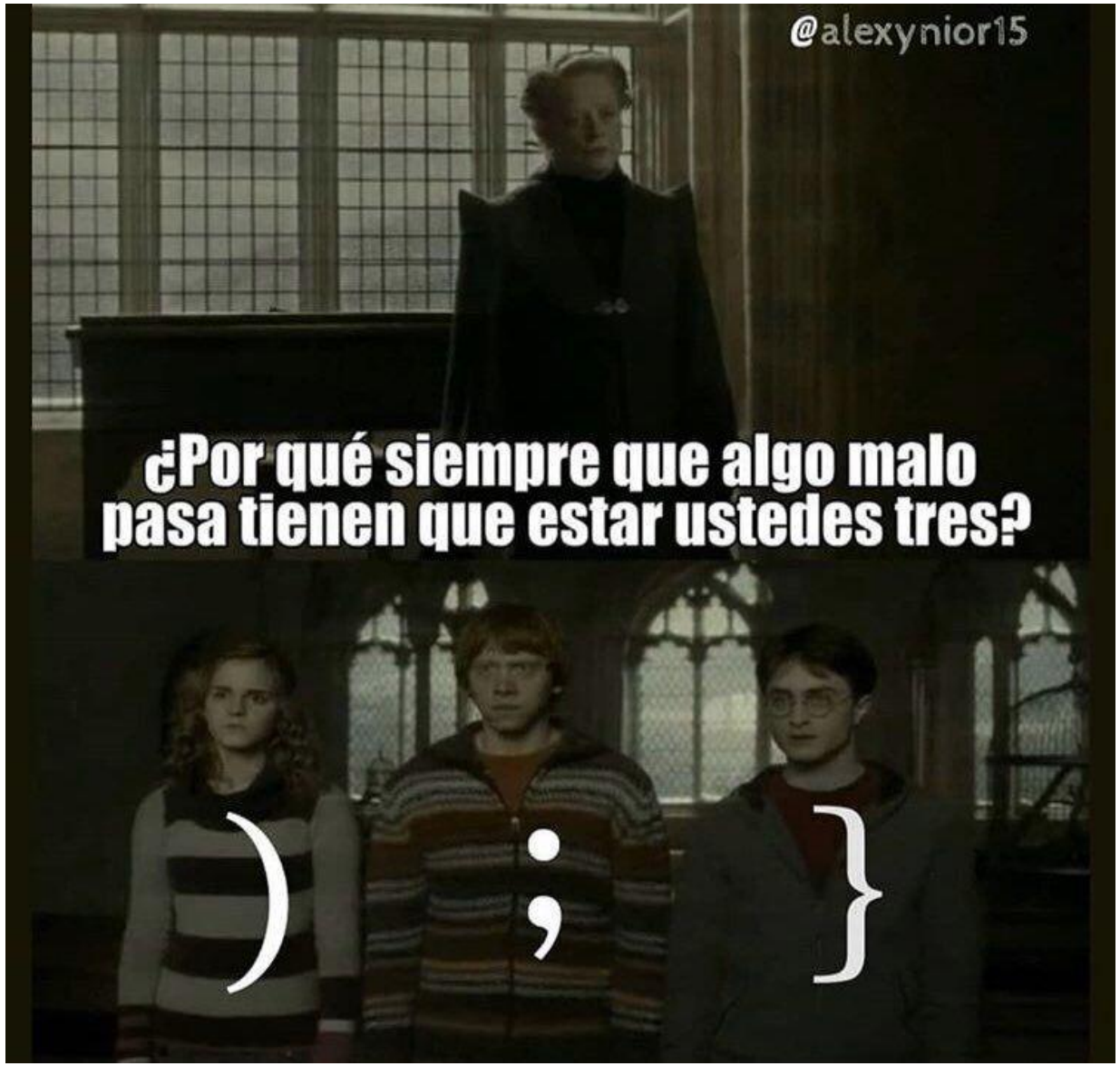
Ejemplo:

✓ una computadora está compuesta de varios componentes como el CPU, la memoria RAM, y el disco duro. Cada uno de estos componentes es esencial para que la computadora funcione, y típicamente, no tienen una función útil por sí mismos fuera de la computadora.

Computadora.java

```
public class Computadora {  no usages
    //Podemos tener un array con la cantidad de discos
     //que tenga la computadora
     Discos[] disco;  no usages

     //Una computadora tiene cierta cantidad de memoria
     //RAM en 1 o varios Slots
     Memoria[] memorias;  no usages
```


@alexynior15
¿Por qué siempre que algo malo pasa tienen que estar ustedes tres?
) ; }

UADE

Síntesis

Síntesis

- Las operaciones entre clases se refieren a las interacciones que pueden tener diferentes clases mediante métodos, donde una clase puede utilizar métodos de otra para ejecutar tareas, facilitando la reutilización de código y la organización lógica del programa. Además, estas operaciones permiten la comunicación y el intercambio de datos entre objetos de diferentes clases para lograr una funcionalidad más compleja.

Síntesis

- La cardinalidad en las relaciones entre clases se refiere a la cantidad de instancias que una clase puede tener de otra en una relación específica. Describe cuántos objetos de una clase pueden estar asociados o dependen de objetos de otra clase, como "uno a uno", "uno a muchos", "muchos a uno" o "muchos a muchos", proporcionando una guía sobre cómo se deben estructurar y vincular los datos en el diseño del *software*.

Síntesis

- La direccionalidad en las relaciones entre clases se refiere a la capacidad de una clase para conocer y utilizar los métodos y atributos de otra clase. Indica si la comunicación entre dos clases es unidireccional, donde solo una clase conoce a la otra y puede invocar sus métodos o acceder a sus atributos, o bidireccional, donde ambas clases se conocen y pueden interactuar entre sí. Esta característica es crucial para el diseño de cómo los objetos se comunican y colaboran en un sistema.

Síntesis

- El término "ordinalidad" no es comúnmente usado. Sin embargo, en áreas relacionadas como bases de datos y modelado de datos, la ordinalidad puede referirse a conceptos como la "opcionalidad" en las relaciones, indicando si la participación de una entidad en una relación es obligatoria u opcional. En la POO, esto podría ser interpretado como si la presencia de un objeto o una clase en una relación es necesaria o si puede ser omitida. Esto determina si un objeto de una clase debe necesariamente estar asociado con un objeto de otra clase o si esa asociación es facultativa.

Bibliografía utilizada para este recurso

GUARDATI, Silvia. Capítulo 2. *Estructuras de datos básicas programación orientada a objetos con JAVA*. México: Alfaomega, 2015. ISBN 9786076224519

UADE

¡Muchas gracias!